

C function examples



make me the content , try not to miss any point from your side.
try to make me visualize and make the function using the C language.
try to solve the examples that may help to understand way further.

Nice—you gave Unit-V from an algorithms syllabus. Below I'll cover **every important point** from that unit, give intuitive visualizations, show working **C code** for the main algorithms (FFT-based polynomial multiplication, randomized QuickSelect, randomized QuickSort, a simple RNG LCG and a verifier for the CLIQUE decision problem), and solve worked examples so you can follow step-by-step.

I'll split this into clear sections. If anything needs deeper detail or more examples, tell me which section and I'll expand.

1) Polynomials and FFT — intuitive idea, complexity, and C implementation

What is the problem?

Given two polynomials $A(x)$ and $B(x)$ with degrees n and m , compute product $C(x) = A(x)B(x)$. Naïve multiplication is $O(nm)$. FFT gives $O(N \log N)$ where N is next power of two $\geq n + m + 1$.

Key idea (visualized):

- A polynomial is determined by its values at enough points. If you evaluate both polynomials at the same N points, multiply pointwise, then interpolate, you get the coefficients of the product.
- FFT fast-evaluates at the N -th roots of unity and inverse-FFT interpolates back.
- Visualization: think of rotating vectors around a circle (complex roots of unity). FFT folds the evaluation recursively (divide even/odd coefficients) until single coefficients, then recombines.

Complexities:

- Time: $O(N \log N)$.
- Space: $O(N)$ extra for arrays.

C implementation (Cooley–Tukey iterative FFT using `<complex.h>`)

This code multiplies two polynomials with double precision complex numbers. It is compact and practical.

c

```
// fft_poly_mul.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <complex.h>

static void fft(complex double *a, int n, int invert) {
    // bit-reversal permutation
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) {
            complex double tmp = a[i]; a[i] = a[j]; a[j] = tmp;
        }
    }

    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * M_PI / len * (invert ? -1 : 1);
        complex double wlen = cos(ang) + sin(ang) * I;
        for (int i = 0; i < n; i += len) {
            complex double w = 1.0 + 0.0*I;
            for (int j = 0; j < len/2; ++j) {
                complex double u = a[i+j];
                complex double v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }
}
```

```

}

if (invert) {
    for (int i = 0; i < n; ++i) a[i] /= n;
}
}

int next_power_of_two(int v) {
    int p = 1;
    while (p < v) p <<= 1;
    return p;
}

void multiply_polynomials(double *A, int n, double *B, int m, double **C_out, int *degC) {
    int result_deg = n + m - 2; // if degrees are n-1 and m-1
    int N = next_power_of_two(n + m - 1);
    complex double *fa = malloc(sizeof(complex double) * N);
    for (int i = 0; i < N; ++i) fa[i] = 0.0 + 0.0*I;
    for (int i = 0; i < n; ++i) fa[i] = A[i];
    for (int i = 0; i < m; ++i) fa[i] += 0; // B will be loaded in fb

    complex double *fb = malloc(sizeof(complex double) * N);
    for (int i = 0; i < N; ++i) fb[i] = 0.0 + 0.0*I;
    for (int i = 0; i < m; ++i) fb[i] = B[i];

    fft(fa, N, 0);
    fft(fb, N, 0);
    for (int i = 0; i < N; ++i) fa[i] *= fb[i];
    fft(fa, N, 1);

    double *C = malloc(sizeof(double) * (result_deg + 1));
    for (int i = 0; i <= result_deg; ++i) {
        C[i] = round(creal(fa[i])); // rounding to nearest integer where appropriate
    }
    *C_out = C;
    *degC = result_deg;

    free(fa);
    free(fb);
}

int main() {

```

```
// Example:  $A(x) = 1 + 2x + 3x^2$  (coeffs [1,2,3])
//       $B(x) = 4 + 5x$  (coeffs [4,5])
double A[] = {1,2,3};
double B[] = {4,5};
int n = 3, m = 2;
double *C; int degC;
multiply_polynomials(A,n,B,m,&C,&degC);
printf("Product polynomial coefficients (constant term first):\n");
for (int i = 0; i <= degC; ++i) printf("%.0f ", C[i]);
printf("\n"); // expect: [4,13,22,15] =>  $4 + 13x + 22x^2 + 15x^3$ 
free(C);
return 0;
}
```

Worked example

$A = [1,2,3]$ ($1 + 2x + 3x^2$), $B=[4,5]$ ($4 + 5x$).

Naïve product:

- x^0 : $1*4 = 4$
- x^1 : $15 + 24 = 5 + 8 = 13$
- x^2 : $25 + 34 = 10 + 12 = 22$
- x^3 : $3*5 = 15$

Output: `4 13 22 15` — code prints that.

2) Probabilistic Algorithms — overview, types, and C functions

Definitions / types

- **Randomized algorithm**: uses random choices. Two main categories:
 - **Las Vegas**: always correct, running time random (e.g., Randomized QuickSelect if we randomize pivot).
 - **Monte Carlo**: deterministic running time, may have a small probability of incorrect answer (e.g., some primality tests).
- **Expected running time** often used in analysis.

Why use randomness?

- Simpler algorithms, better average-case behaviors, or performance guarantees that are good with high probability.

Randomized QuickSort (idea & complexity)

- Choose pivot uniformly at random from subarray; expected time $O(n \log n)$. Worst-case $O(n^2)$ but probability tiny.
- Space: $O(\log n)$ expected recursion stack.

C snippet (in-place randomized QuickSort)

```
c

// rand_quicksort.c (core functions)
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b){ int t=*a; *a=*b; *b=t; }

int partition(int arr[], int l, int r) {
    int pivot = arr[r];
    int i = l - 1;
    for (int j = l; j <= r-1; ++j) {
        if (arr[j] <= pivot) {
            ++i; swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i+1], &arr[r]);
    return i+1;
}

int random_partition(int arr[], int l, int r) {
    int p = l + rand() % (r - l + 1);
    swap(&arr[p], &arr[r]);
    return partition(arr,l,r);
}

void rand_quicksort(int arr[], int l, int r) {
    if (l < r) {
        int q = random_partition(arr,l,r);
    }
}
```

```

    rand_quicksort(arr,l,q-1);
    rand_quicksort(arr,q+1,r);
}
}

int main() {
    srand((unsigned)time(NULL));
    int arr[] = {3,6,8,10,1,2,1};
    int n = sizeof(arr)/sizeof(arr[0]);
    rand_quicksort(arr,0,n-1);
    for (int i=0;i<n;++i) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}

```

Worked intuition: randomly chosen pivots tend to split arrays well on average $\rightarrow O(n \log n)$.

Randomized Selection (QuickSelect) — Las Vegas example + code

Goal: find k-th smallest element in expected linear time $O(n)$.

Idea: Randomly pick pivot, partition, recurse on the side containing k.

```

c

// rand_select.c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b){ int t=*a; *a=*b; *b=t; }

int partition(int arr[], int l, int r) {
    int x = arr[r], i = l;
    for (int j = l; j < r; ++j) {
        if (arr[j] <= x) { swap(&arr[i], &arr[j]); ++i; }
    }
    swap(&arr[i], &arr[r]);
    return i;
}

```

```

int random_partition(int arr[], int l, int r) {
    int idx = l + rand() % (r - l + 1);
    swap(&arr[idx], &arr[r]);
    return partition(arr, l, r);
}

int randomized_select(int arr[], int l, int r, int k) {
    if (l == r) return arr[l];
    int q = random_partition(arr, l, r);
    int left_count = q - l + 1;
    if (k == left_count) return arr[q];
    else if (k < left_count) return randomized_select(arr, l, q-1, k);
    else return randomized_select(arr, q+1, r, k - left_count);
}

int main() {
    srand((unsigned)time(NULL));
    int arr[] = {7, 10, 4, 3, 20, 15};
    int n = sizeof(arr)/sizeof(arr[0]);
    int k = 3; // 3rd smallest -> result should be 7
    int res = randomized_select(arr, 0, n-1, k);
    printf("k-th smallest = %d\n", res);
    return 0;
}

```

Worked example: For array `[7, 10, 4, 3, 20, 15]` , $k=3 \rightarrow$ sorted `[3, 4, 7, 10, 15, 20]` $\rightarrow$ 3rd smallest=7.

3) Algorithms for Random Number Generation

Simple classical generator: Linear Congruential Generator (LCG)

Formula: $X_{n+1} = (aX_n + c) \bmod m$.

- Choose modulus m , multiplier a , increment c , seed X_0 .
- Period $\leq m$; certain parameter choices give maximal period.

C implementation (simple LCG):

```

c

```

```

// lcg.c
#include <stdio.h>
#include <stdint.h>

typedef uint64_t u64;
static u64 state = 1;
u64 lcg_next() {
    const u64 a = 6364136223846793005ULL; // example multiplier
    const u64 c = 1442695040888963407ULL; // example increment
    state = a * state + c;
    return state;
}

void seed_lcg(u64 s) { state = s; }

int main() {
    seed_lcg(12345);
    for (int i = 0; i < 10; ++i) {
        printf("%llu\n", (unsigned long long)(lcg_next()));
    }
    return 0;
}

```

Notes:

- LCG is simple but has statistical weaknesses; modern RNGs use better algorithms (xorshift, xoroshiro, PCG).
- For cryptographic needs, use cryptographically secure RNGs (not LCG).

4) Basic Concepts: NP, NP-hard, NP-complete, Cook's Theorem (statement only)

Definitions

- **P**: set of decision problems solvable in polynomial time by a deterministic Turing machine.
- **NP**: set of decision problems where a "yes" answer has a polynomial-size certificate verifiable in polynomial time.
- **NP-hard**: a problem H is NP-hard if every problem in NP reduces to H (i.e., H is at least as hard as every NP problem).

- **NP-complete:** a problem that is in NP and NP-hard.

Cook's Theorem (statement, without proof)

The Boolean satisfiability problem (SAT) is NP-complete — specifically, any problem in NP can be polynomial-time reduced to SAT. This establishes the first NP-complete problem and is a foundation for NP-completeness theory.

Why reductions matter: show one problem is as hard as another. If you can polynomially reduce known NP-complete problem to a new problem, the new problem is NP-hard. If it belongs to NP as well, it is NP-complete.

5) Reduction — concept and an example with CLIQUE

Reduction (intuition): transform instance of problem A into instance of problem B in polynomial time such that answers correspond.

CLIQUE problem (decision version):

- Input: graph $G = (V, E)$ and integer k .
- Question: Does G contain a complete subgraph (clique) of size at least k ?

Important simple reduction: CLIQUE $\leftrightarrow$ INDEPENDENT SET via complement graph.

- G has a clique of size k iff complement graph $\overline{G}$ has an independent set of size k .
- This is an easy polynomial-time transformation: build complement adjacency in $O(n^2)$.

Verifier for CLIQUE: Given graph and set S of k vertices, verifying clique is polynomial: check all $\binom{k}{2}$ edges present. So CLIQUE is in NP.

C program: verify whether a given vertex set is a clique (certificate verifier)

```
c

// clique_verify.c
#include <stdio.h>
#include <stdlib.h>

int is_clique(int n, int adj[n][n], int *S, int k) {
    for (int i = 0; i < k; ++i) {
```

```

    for (int j = i+1; j < k; ++j) {
        if (!adj[S[i]][S[j]]) return 0; // edge missing
    }
}
return 1;
}

int main() {
    // Example graph adjacency matrix (0-based indices). n=5
    int n = 5;
    int adj[5][5] = {
        {0,1,1,0,0},
        {1,0,1,1,0},
        {1,1,0,0,0},
        {0,1,0,0,1},
        {0,0,0,1,0}
    };
    // Candidate set S = {0,1,2} -> should be clique as edges exist among 0-1,0-2,1-2
    int S[] = {0,1,2};
    int k = 3;
    if (is_clique(n,adj,S,k)) printf("S is a clique of size %d\n", k);
    else printf("S is NOT a clique\n");
    return 0;
}

```

Time/space:

- Verifier time $O(k^2)$.
- CLIQUE decision problem is NP-complete (classic result; often shown via reduction from 3-SAT, but we only state facts here).

6) Cook's Theorem (intuition without proof)

- **Intuition:** Any nondeterministic polynomial time computation can be encoded as a Boolean formula that is satisfiable iff the machine accepts. So SAT captures NP. Cook's Theorem formalizes this.
- We won't prove it here (proof constructs variables representing machine states, tape contents, time steps and constraints to ensure a valid accepting computation).

7) Worked reduction example: CLIQUE $\leftrightarrow$ INDEPENDENT SET (construct complement)

Given graph G :

- Build $\overline{G}$: for all pairs (u,v) , if edge in G then not in $\overline{G}$ and vice versa (except self-loops).
- Then G has clique size $k \Leftrightarrow \overline{G}$ has independent set size k .

Small concrete example:

G with edges: $(0,1),(0,2),(1,2),(1,3),(3,4)$

- G has clique $\{0,1,2\}$ of size 3.
 - Complement has no edges among $0,1,2$ so $\{0,1,2\}$ is independent in complement? Actually complement edges are all non-edges of G , so in complement $0,1,2$ are pairwise non-adjacent $\rightarrow$ independent set of size 3.
-

8) Additional examples & solved problems

Example 1: Multiply polynomials using FFT (we already did) — result explained.

Example 2: Find 3rd smallest element using randomized select (worked above: result 7).

Example 3: Verify a clique (worked above).

Example 4: Randomized QuickSort demonstration (small array)

- Input: `[3, 6, 8, 10, 1, 2, 1]`.
 - After random pivot choices and partitioning, you end up sorted. Expectation of comparisons $\sim O(n \log n)$.
-

9) Complexity summary (Unit-V algorithms)

- FFT polynomial multiplication: $O(N \log N)$ time, $O(N)$ space.
- Randomized QuickSort: expected $O(n \log n)$ time, worst $O(n^2)$.
- Randomized QuickSelect: expected $O(n)$, worst $O(n^2)$.
- LCG RNG: $O(1)$ per number, space $O(1)$.
- Clique verification (given certificate): $O(k^2)$.

- NP-complete membership: no known polynomial-time algorithm (open problem P vs NP).
-

10) Visual tips to remember & imagine

- **FFT:** picture splitting coefficients into even/odd (two smaller polynomials), like splitting a line into two and then folding them back — recursively until singletons, then recombine waves (complex roots) with butterfly operations.
 - **Randomized algorithms:** imagine random pivots as “random knives” cutting the array — most of the time they cut near the middle and produce good subproblems.
 - **Reduction:** imagine transforming a puzzle board A into a puzzle board B using a recipe. If B is hard and every A can be turned into a B, B must be at least as hard.
-

If you want:

- I can run and test any of these C programs on some inputs and return compiled output (I can produce code files for you).
- Or I can expand one area: e.g., a detailed step-by-step of FFT butterfly operations on a specific input, or a Miller-Rabin primality test as Monte Carlo example, or show the classic reduction 3-SAT → CLIQUE sketch.

Which of the above would you like me to expand first? (FFT step-by-step diagram, Miller-Rabin code, or reduction 3-SAT→CLIQUE sketch?)